

This document is designed to be handed directly to a Technical Lead or Lead iOS Developer. It outlines the specific technical requirements and architectural constraints necessary to build a professional-grade AR visualizer.

Technical Specifications: Decorative Wall Art Visualizer

1. Project Overview

The objective is a high-fidelity iPad application that allows users to project decorative wall textures/styles onto physical walls in real-time or via photo capture. The primary focus is **visual accuracy**, specifically regarding perspective, lighting, and occlusion.

2. Core Technical Stack

- **Platform:** iOS/iPadOS (Targeting iPadOS 16+).
- **Frameworks:** ARKit 6 (Scene Reconstruction), RealityKit (Rendering), Metal (Post-processing if needed).
- **Hardware Minimum:** A12 Bionic chip; **LiDAR recommended** (iPad Pro 2020+ or iPad Air latest).

3. Feature Requirements & Implementation Logic

A. Surface Detection & Plane Mapping

The builder must implement a "Coaching Overlay" to guide the user in scanning the room.

- **Vertical Plane Detection:** Enable `ARWorldTrackingConfiguration` with `.vertical plane detection`.
- **Mesh Generation:** If LiDAR is available, use `sceneReconstruction = .meshWithClassification` to identify walls specifically vs. floors.
- **Visual Feedback:** Implement a subtle "shimmer" or grid effect on detected walls to indicate to the user where the art can be applied.

B. Intelligent Occlusion (The "Depth" Problem)

To prevent the art from "floating" over furniture:

- **Scene Depth:** Use `frame.sceneDepth` to create a depth map.
- **Object Occlusion:** Enable `reconstruction.occlusion` in RealityKit. This ensures that if a sofa is in front of the wall, the decorative style appears behind it.
- **Segmentation:** Use `ARFrame.SegmentationBuffer` to mask out humans in real-time.

C. Material & Texture Application

The "styles" (30+ assets) must be applied as materials to the detected surface.

- **PBR Materials:** Styles should be implemented as **Physically Based Rendering** materials. Do not use flat images; provide maps for:
 - *Albedo/Base Color* (The design).
 - *Roughness/Metallic* (To define if the wall art is matte or glossy).
 - *Normal Maps* (To simulate the texture of the art/wallpaper).
- **Tiling Logic:** Implement a "UV Scaling" feature allowing the user to adjust the scale of the pattern once applied to the wall.

D. Dynamic Lighting & Environment

- **Light Estimation:** Use `AREnvironmentProbeAnchor`. This will sample the room's actual light and apply those reflections and shadows to the digital wall style.
- **Shadow Casting:** Ensure the digital surface can receive shadows from physical objects (using `ShadowReceiver` components in `RealityKit`).

4. UI/UX Flow for Developers

1. **Scanning State:** User moves iPad; app highlights walls in real-time.
2. **Selection State:** User taps a highlighted wall.
3. **Application State:** Selected style is mapped to the wall boundary.
4. **Adjustment State:** User can pinch-to-zoom (scale pattern) or drag (reposition pattern).
5. **Capture State:** High-resolution snapshot functionality that flattens the AR view and UI into a shareable JPEG.

5. Asset Management Requirement

- **Format:** Textures should be provided in 2K or 4K resolution.
- **Optimization:** Use **USDZ** containers for any 3D elements or standardized **MaterialX** for complex shaders.
- **Loading:** Implement an asynchronous loading pattern to prevent the UI from freezing when switching between the 30 different styles.

6. Success Metrics for Delivery

- **Persistence:** The style should stay "stuck" to the wall even if the user walks around the room.
- **Latency:** Style switching should happen in under 500ms.
- **Accuracy:** The digital texture must respect corners and trim (molding) via accurate plane clipping.

Would you like me to draft a "User Story" document to accompany this, which explains the experience from the perspective of your sales reps?